

Dijkstra 邻接表+优先队列(最小堆) $E\log V$

邻接矩阵 + 不用优先队列

每次都扫描所有点找最小 `dist`:

$O(V^2)$

: 邻接表 + 优先队列 / 最小堆

每条边最多被松弛，优先队列操作是 $\log V$ 。

复杂度一般写:

$O((V+E)\log V)$

也经常简化成:

$O(E\log V)$

适合稀疏图。

数组

`int[] dist;`

`int[] prev;/ parent[]`

`boolean[] visited;`

`dist[x]` 表示从源点 `source` 到顶点 `x` 当前已知的最短距离

`prev[x]` 表示当前最短路径中, 走到 `x` 之前的那个点是谁 这个数组的作用是**恢复路径**

`visited[x] = true` 表示顶点 `x` 的最短距离已经最终确定

`void dijkstra(Graph G, int source) {`

`int[] dist = new int[G.V()];`

`int[] prev = new int[G.V()];`

```

boolean[] visited = new boolean[G.V()];

Arrays.fill(dist, INF);

Arrays.fill(prev, -1);

dist[source] = 0;

PriorityQueue<Node> pq = new PriorityQueue<>(
    (a, b) -> a.distance - b.distance
);

pq.add(new Node(source, 0));

while (!pq.isEmpty()) {
    Node current = pq.poll();

    int u = current.vertex;

    if (visited[u]) {
        continue;
    }

    visited[u] = true;

    for (Edge edge : G.adj(u)) {
        int v = edge.to;

        int w = edge.weight;

        if (!visited[v] && dist[u] + w < dist[v]) {
            dist[v] = dist[u] + w;

            prev[v] = u;
        }
    }
}

```

```

        pq.add(new Node(v, dist[v]));
    }
}
}
}
}

```

迪杰斯特拉主循环递归

```

void dijkstraRecursive(PriorityQueue<Node> pq) {
    if (pq.isEmpty()) {
        return;
    }
    Node cur = pq.poll();
    int u = cur.vertex;
    if (visited[u]) {
        dijkstraRecursive(pq);
        return;
    }
    visited[u] = true;
    for (Edge edge : adj[u]) {
        int v = edge.to;
        int w = edge.weight;

        if (!visited[v] && dist[u] + w < dist[v]) {
            dist[v] = dist[u] + w;
            prev[v] = u;
        }
    }
    pq.add(cur);
}

```

```

        pq.add(new Node(v, dist[v]));
    }

}

dijkstraRecursive(pq);
}

```

问题是递归层数可能很多。

如果 `priority queue` 里有很多元素，递归可能很深 `StackOverflowError` 容易

Prim 邻接表+优先队列 $O(E \log V)$

每一次操作检查点并且把边放到优先队列里操作加起来 $E \log V$ 来取最小

```

class Edge {

    int from;

    int to;

    int weight;

}

```

```

ArrayList<Edge> prim(ArrayList<Edge>[] adj, int start) {

    boolean[] visited = new boolean[adj.length];

    ArrayList<Edge> mst = new ArrayList<>();
}

```

```

PriorityQueue<Edge> pq = new PriorityQueue<>(

    (e1, e2) -> e1.weight - e2.weight

);

visited[start] = true;

for (Edge e : adj[start]) {

    pq.add(e);

}

while (!pq.isEmpty() && mst.size() < adj.length - 1) {

    Edge e = pq.poll();

    if (visited[e.from] && visited[e.to]) {

```

```

        continue;

    }

    mst.add(e);

    int newVertex;

    if (!visited[e.from]) {

        newVertex = e.from;

    } else {

        newVertex = e.to;

    }

    visited[newVertex] = true;

    for (Edge next : adj[newVertex]) {

        if (!(visited[next.from] && visited[next.to])) {

```

```

        pq.add(next);

    }

}

return mst;

}

```

名字	类型	含义
visited[]	boolean[]	记录每个点是否已经加入 MST
adj[]	ArrayList<Edge>[]	邻接表，记录每个点连接的边
mst	ArrayList<Edge>	保存最终选入 MST 的边
pq	PriorityQueue<Edge>	候选边集合，每次取最小权重边
Edge	class/object	表示一条边：起点、终点、权重

Prim 递归

Prim(G, start):

```

visited[start] = true

mst = empty list

pq = empty priority queue

for each edge e from start:

    pq.add(e)

PrimRecursive(G, visited, pq, mst)

return mst

```

```

PrimRecursive(G, visited, pq, mst):

    if pq is empty:

        return

    if mst.size == V - 1:

        Return

    e = pq.removeMin()

    if both endpoints of e are visited:

        PrimRecursive(G, visited, pq, mst)

        return

    add e to mst

    newVertex = the endpoint of e that is not visited

    visited[newVertex] = true

    for each edge nextEdge from newVertex:

        if nextEdge connects to an unvisited vertex:

            pq.add(nextEdge)

    PrimRecursive(G, visited, pq, mst)

```

Kruskal 边列表+union 0 ($E \log E$)

边排序 $E \log E$ union 操作 $O(1)$

Edge() 储存边信息 别的临界列表储存该点相邻点

Kruskal(G):

```

mst = empty list

sort all edges by increasing weight

```



```

create UnionFind for all vertices

for each edge (u, v, weight) in sorted edges:

    if find(u) != find(v):

        add edge to mst

        union(u, v)

    if mst.size == V - 1:

        break

return mst

```

关键操作 `if find(u) != find(v)`: 如果 `u` 和 `v` 不在一个集合下就不能成环可以合并

名字	类型	含义
<code>edges[]</code> / <code>edgeList</code>	边列表	存图里所有边
<code>parent[]</code>	并查集数组	记录每个点的父节点 / 所属集合
<code>rank[]</code> 或 <code>size[]</code>	并查集优化数组	控制合并方向, 让树更矮
<code>mst[]</code> / <code>result</code>	结果边列表	存最后选进 MST 的边

递归 `Kruskal(G)`:

```

edges = all edges of G

sort edges by increasing weight

uf = new UnionFind(V)

mst = empty list

KruskalRecursive(edges, 0, uf, mst)

return mst

```

`KruskalRecursive(edges, index, uf, mst)`:

```

if mst.size == V - 1:

```

```

        return

    if index == edges.length:

        return

    edge = edges[index]

    u = edge.from

    v = edge.to

    if uf.find(u) != uf.find(v):

        mst.add(edge)

        uf.union(u, v)

    KruskalRecursive(edges, index + 1, uf, mst)

```

BST 二叉搜索树 Node 节点, 通过 left/right 引用连起来

节点里 Key left right 实现插入删除查询

操作	平衡 BST	最坏 BST
Search	$O(\log n)$	$O(n)$
Insert	$O(\log n)$	$O(n)$
Delete	$O(\log n)$	$O(n)$
Inorder traversal	$O(n)$	$O(n)$

查询

```

if (key < node.key) {
    return contains(node.left, key);
} else if (key > node.key) {
    return contains(node.right, key);
} else {
    return true;
}

```

插入 比较 key 走左

$6 < 8$, 去左边

$6 > 3$, 去右边

右边是 null, 插入这里 插到叶子里

删除三种情况

1. 删除叶子节点 没孩子直接删掉

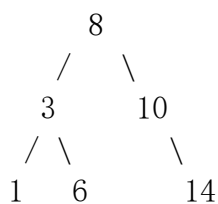
2. 删除只有一个孩子的节点 删除父把孩子顶上来

```
if (node.left == null) {  
    return node.right;  
}
```

```
if (node.right == null) {  
    return node.left;  
}
```

3. 删除有两个孩子的节点

找右子树中最小的节点 successor, 用它替代当前节点



BST 复杂度与树高有关系

$O(h)$ 树平衡 $h = \log n$ $O(\log n)$

2-3 树

操作	时间复杂度
Search	$O(\log n)$
Insert	$O(\log n)$
Delete	$O(\log n)$
Traversal	$O(n)$

空间复杂度 $O(n)$

对比	BST	2-3 树
一个节点几个 key	1 个	1 个或 2 个
一个节点几个孩子	最多 2 个	2 个或 3 个
是否自动平衡	普通 BST 不会	会
所有叶子是否同一层	不一定	一定
最坏复杂度	$O(n)$	$O(\log n)$

一种永远保持平衡的搜索树。它是 BST 的升级版, 目的就是避免 BST 退化成链表 **2-3 树** 会自动分裂、合并, 保持所有叶子在同一层, 所以查找、插入、删除都是: $O(\log n)$

平衡原因：它只允许在叶子插入，并且溢出时把中间值往上推。只有根分裂时，整棵树高度才一起增加。

搜索

Search(node, key):

 if node is null:

 return false

 if key equals one of node's keys:

 return true

 if node is 2-node:

 if key < node.key:

 return Search(node.left, key)

 else:

 return Search(node.right, key)

 if node is 3-node:

 if key < node.key1:

 return Search(node.left, key)

 else if key < node.key2:

 return Search(node.middle, key)

 else:

 return Search(node.right, key)

插入

Insert(key):

find the leaf where key should go

insert key into that leaf

while current node has 3 keys:

split current node

promote middle key to parent

if parent becomes overfull:

current = parent

else:

stop

if root is overfull:

split root and create new root

修复方法一：向兄弟借 key

如果兄弟节点有两个 key，可以借一个。

比如：

```
    20
   /  \
空  30 | 40
```

右兄弟有两个 key，可以借。

修复后大概变成：

```
    30
   /  \
20    40
```

意思是：

父节点的 key 下来
兄弟的一个 key 上去

这种叫 rotation / redistribution。

10. 修复方法二：和兄弟合并

如果兄弟也只有一个 key，借不了，就合并。

比如：

```
      20
     /  \
    空   30
```

可以把父节点 20 和兄弟 30 合并：

20 | 30

父节点少了一个 key。

如果父节点也变空，就继续向上修复。

所以删除可能一路向上合并，像往上收缩的拉链 。

11. 删除内部节点怎么办？

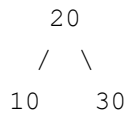
如果删除的 key 在内部节点，不在叶子里，通常先替换成：

前驱 predecessor

或者：

后继 successor

比如删除内部节点的 20：



可以用左子树最大值 10 替代 20，然后再去叶子删掉 10。

这和 BST 删除两个孩子节点很像。

12. 删除伪代码

概念版：

```

Delete(key):
    find key

    if key is in internal node:
        replace it with predecessor or successor
        delete predecessor/successor from leaf

    delete key from leaf

    while current node has no key:
        if sibling has 2 keys:
            borrow from sibling through parent
            stop
        else:
            merge current node with sibling and parent key
            current = parent

    if root has no key:
        make its only child the new root

```

考试一般不要求你手写完整删除代码，更多是理解：

删了以后不够，就借；借不了，就合并。

红黑树 红不能连红

红黑树 Red-Black Tree, 它其实是 一种自平衡二叉搜索树, 可以看作是 2-3 树的二叉树实现版本

操作	平均/最坏复杂度
查找 Search	$O(\log n)$
插入 Insert	$O(\log n)$
删除 Delete	$O(\log n)$

操作 平均/最坏复杂度
遍历 Traversal ($O(n)$)

```
class Node {  
    int key;  
    Node left;  
    Node right;  
    Node parent;  
    boolean color; // true=红, false=黑  
}
```

红黑树必须满足的 5 条性质

每个节点要么红，要么黑

颜色属性保证平衡调节的依据。

根节点必须是黑色

根黑保证路径上至少有一个黑节点。

所有叶子（NIL 节点）是黑色

空节点也算黑色，用来计算黑高。

红节点的子节点必须是黑色

红不能连红，避免路径太长。

从任意节点到叶子的每条路径包含相同数量的黑色节点

这个黑高规则保证了树高度不会超过 $2\log n$

红黑树和 BST 的关系

- **BST** 保持左小右大
- **红黑树也是 BST**，只是在插入/删除时加了颜色和旋转来平衡高度。

所以它既可以做普通 **BST** 的查找/插入/删除，又保证最坏时间复杂度： $O(\log n)$

高度保证

因为红色不能连红，并且每条路径黑节点数相同：

- 最短路径全是黑节点 $\rightarrow$ 长度 = 黑高 = h_{black}
- 最长路径红黑交替 $\rightarrow$ 长度 $\leq 2 \times$ 黑高

所以高度 $\leq 2 \times \log(n+1)$ ，查找/插入/删除最坏也是 $O(\log n)$

插入操作

插入步骤：

1. 按 BST 插入新节点，**初始颜色红**。
2. 检查是否违反性质 4（红不能连红）：
 1. 如果父节点是黑色 $\rightarrow$ 不用调整
 2. 如果父节点是红色 $\rightarrow$ 可能需要旋转和重新着色
3. 旋转和着色规则（几种情况）：
 1. **父红，叔红** $\rightarrow$ 父/叔变黑，祖变红 $\rightarrow$ 递归向上修复
 2. **父红，叔黑** $\rightarrow$ 做旋转（左/右） + 重新着色

旋转操作：

左旋 (Left-Rotate)

右旋 (Right-Rotate)

它保证：

- BST 结构不变
- 红黑性质恢复

删除操作

删除稍复杂：

1. 如果删除的是红节点 $\rightarrow$ 直接删除，不破坏黑高
2. 如果删除的是黑节点 $\rightarrow$ 破坏黑高，需要 **双黑修复**：
 1. 和兄弟节点调整颜色
 2. 可能旋转和向上递归

最终保证：

- 红黑性质恢复
- 树高度 $\leq 2 \log n$

红黑树和 2-3 树的关系

- 红黑树是 2-3 树的二叉化实现：
 - **2-node** → 黑节点
 - **3-node** → 黑节点 + 红色子节点
- 所以红黑树插入删除通过颜色和旋转模拟 2-3 树分裂/合并。
- 叶子高度平衡自动保证最坏查找仍是 $O(\log n)$ 。

红黑树 = BST + 颜色 + 旋转 + 平衡性质

颜色 + 红黑规则 + 旋转相当于把 2-3 树的分裂/合并操作“翻译成二叉树”，自动保证最坏高度 $O(\log n)$ ，查找/插入/删除都是对数时间

BFS 临接列表+队列 $O(V+E)$

数组名称

数组名称	类型	作用
visited 或 marked	boolean[V]	防止重复访问
distance	int[V]	记录最短路径长度（可选）
parent	int[V]	记录路径前驱（可选）

队列适合用 while 循环当队列不空就继续

BFS(G, s):

```
queue = empty queue
```

```
visited = array of size  $|V|$ , all false
```

```
visited[s] = true
```

```
enqueue(queue, s)
```

```
while queue is not empty:
```

```
    v = dequeue(queue)
```

```
    visit(v)  # 处理节点 v，比如打印或者记录路径
```

```
    for each neighbor w of v in G.adj(v):
```

```
        if not visited[w]:
```

```
            visited[w] = true
```

```
            enqueue(queue, w)
```

递归版本 就是把 while 换成了递归 base 队列空了 return 把一个点的邻居都访问过之后返回递归队列处理队列其他点的邻居

```
BFS(G, start):
```

```
    queue = empty queue
```

```
    visited[start] = true
```

```
    queue.add(start)
```

```
    BFSRecursive(G, queue, visited)
```

递归核心

```
BFSRecursive(G, queue, visited):
```

```
    if queue is empty:
```

```
        return
```

```
    v = queue.remove()
```

```
    visit(v)
```

```
    for each neighbor w of v:
```

```
        if not visited[w]:
```

```
            visited[w] = true
```

```
            queue.add(w)
```

```
BFSRecursive(G, queue, visited)
```

DFS 邻接表+stack $O(V+E)$

递归天然栈

```
DFS_Recursive(G, v, visited):
```

```
    visited[v] = true
```

```
    visit(v) # 处理节点，例如打印或者记录
```

```
    for each neighbor w of v in G.adj(v):
```

```
    if not visited[w]:  
        DFS_Recursive(G, w, visited)
```

显示栈

```
DFS_Stack(G, start):  
  
    stack = empty stack  
  
    visited = array of size V, all false  
  
    stack.push(start)  
  
    while stack is not empty:  
        v = stack.pop()  
  
        if not visited[v]:  
            visited[v] = true  
            visit(v) # 处理节点  
  
            # 将邻居压入栈  
            for each neighbor w of v in G.adj(v):  
                if not visited[w]:  
                    stack.push(w)
```

递归的优点和缺点

递归是一种函数调用自身来解决问题的方法，常用于把一个复杂问题拆分成更小、结构相似的子问题。它的优点是代码通常比较简洁，逻辑清晰，特别适合分治算法、树结构、图的深度优先搜索以及回溯问题。例如归并排序、快速排序、二叉树遍历、DFS、汉诺塔等问题，用递归表达会更加自然。

递归的另一个优点是它和很多数学定义非常接近。例如阶乘可以写成 $n! = n \times (n-1)!$ ，斐波那契数列也可以用递归关系表示。因此，在处理这类问题时，递归能让程序结构更容易理解。

但是，递归也有一些缺点。首先，每一次递归调用都会占用函数调用栈空间，如果递归层数太深，可能会导致栈溢出。其次，递归可能带来额外的时间开销，因为函数调用本身需要保存参数和返回地址。如果递归中出现大量重复计算，而且没有使用记忆化或动态规划优化，程序效率会很低。例如普通递归计算斐波那契数列就会产生很多重复子问题。

总的来说，递归适合用于问题可以自然分解成相同类型子问题的情况，例如分治、树遍历、DFS 和回溯算法。但对于简单循环、大规模线性遍历，或者递归深度可能非常大的问题，使用迭代方法通常更安全、更高效。递归的核心优势是清晰和自然，缺点是可能带来额外空间开销和效率问题。

Sorting 递归思想

Bubble 递归

递归版冒泡排序的核心是：

把问题分解成两个部分：

1. 把数组中最大的元素移动到最后（一轮冒泡）
2. 递归地对剩下的 $n-1$ 个元素排序

也就是说，递归就是迭代外层循环的替代

BubbleSortRecursive(A, n):

```
if n == 1:           # 基本情况，只有一个元素

    return

# 一轮冒泡，把最大元素放到最后

for i = 0 to n-2:

    if A[i] > A[i+1]:

        swap(A[i], A[i+1])

# 递归对前 n-1 个元素排序

BubbleSortRecursive(A, n-1)
```

冒泡排序可以用递归实现。其思想是先进行一轮冒泡，把当前未排序部分的最大元素移动到最后，然后递归地对前 $n-1$ 个元素继续排序。这样写可以体现递归中“把大问题变成小问题”的思想，逻辑上比较清楚。

但是，递归实现冒泡排序的实际优点并不明显。它不会降低时间复杂度，仍然是 $O(n^2)$ 。同时，递归调用会占用额外的栈空间，使空间复杂度从循环版本的 $O(1)$ 增加到 $O(n)$ 。当数据规模较大时，还可能出现栈溢出。因此，递归版冒泡排序更适合用于理解递归思想，而不适合实际高效排序。实际编程中，冒泡排序通常使用循环实现。

MergeSort 递归

分开用递归合起来用 while left right 不空

这两句的意思不是“直接得到排好的 left/right”，而是：

- 对左半边继续调用 MergeSort
- 对右半边继续调用 MergeSort
- 一直调用到长度为 1

分用 MergeSort

```
MergeSort(arr):
if length(arr) <= 1:
return arr # 已经有序
mid = length(arr) // 2
# 分别递归对左右半部分排序
left = MergeSort(arr[0:mid])
right = MergeSort(arr[mid:])

# 合并两个有序数组
return Merge(left, right) 合用 merge 比较 left 和 right 谁小谁放到 result 里
Merge(left, right):
result = empty array
i = 0
j = 0
# 当左右都有元素时，比较当前最小值
while i < length(left) and j < length(right):
if left[i] <= right[j]:
result.append(left[i])
i = i + 1
else:
result.append(right[j])
j = j + 1
# 把左边剩余元素放进结果
while i < length(left):
result.append(left[i])
i = i + 1
# 把右边剩余元素放进结果
while j < length(right):
result.append(right[j])
j = j + 1
return result
```


选择排序递归

```
RecursiveWeirdSort(array, i):  
    n = length(array)  
  
    # 递归结束条件: i 到最后一个位置  
    if i >= n - 1://前面排好了最后一个可以不用排序  
        return  
  
    # 找出 i 之后最小(或最大的)元素并交换到 i  
    for j = i+1 to n-1:  
        if array[j] < array[i]:  
            swap(array[j], array[i])  
  
    # 递归处理下一个位置  
    RecursiveWeirdSort(array, i+1)
```

选择排序

```
InsertionSortRecursive(A, n):  
    # 基本情况: 只有一个元素或者没有元素, 已经有序  
    if n <= 1:  
        return  
  
    # 先递归排序前 n-1 个元素  
    InsertionSortRecursive(A, n-1)  
  
    # 把第 n 个元素插入到前 n-1 个已经排好序的数组里  
    key = A[n-1]  
    j = n - 2  
  
    # 向右移动大于 key 的元素  
    while j >= 0 and A[j] > key:  
        A[j+1] = A[j]  
        j = j - 1  
  
    # 放置 key  
    A[j+1] = key
```